Report of Thread-Safe Malloc Project

Name: Ziyu Shi NetID: zs148

In this report, I will explain my implementation of my two versions of thread-safe malloc library and their corresponding free() with C, and will conduct a performance experimentation and analysis on my two versions of thread-safe malloc library.

1. An overview of my implementation

In this section, I will elaborate my implementation of these two versions of thread-safe malloc library based on the Best Fit function realized in the Project 1. I set different free head block pointers and free tail block pointers for different versions of thread-safe malloc. The race conditions that lead to incorrect results are usually caused by concurrent modifications of critical sections accessed by multi-threads. **To solve this, I decided to assign unique linklist of free blocks to every thread.** So, every thread can only manipulate their own linklist of free blocks. The implementation is shown below.

```
// head and tail for locked safe malloc
Metadata * first_free_lock_blk = NULL;
Metadata * last_free_lock_blk = NULL;

// head and tail for unsafe malloc in every seperate thread
__thread Metadata * first_free_unlock_blk = NULL;
__thread Metadata * last_free_unlock_blk = NULL;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

first_free_lock_blk and **last_free_lock_blk** are used in the **locking version**. In this version, **ts_malloc_lock will use pthread_mutex_lock lock the entire process of bf_malloc**, which means there is no possibility that multi-threads manipulate one free block.

first_free_unlock_blk and **last_free_unlock_blk** are used in unlocking version and are assigned to every thread, **which means that every thread has its own head pointer and tail pointer of free blocks list**. In this version, I need to **use pthread_mutex_lock in allocate_block() to ensure that only one thread can call for allocation at one time.**

To differ allocate_block() in unlocking version from the one in locking version, I set **sbrk_lock** variable. If sbrk_lock is 0, the current version of allocate_block() is locking version, else if sbrk_lock is 1, the current version is unlocking version. The allocate_block() is shown below.

```
void * allocate_block(size_t size, int sbrk_lock) {
  Metadata * new_block = NULL;
  // sbrk_lock == 0: this is thread safe malloc
  if(sbrk_lock == 0) {
    new_block = sbrk(size + sizeof(Metadata));
  }
  // sbrk_lock = 1: this is thread unsafe malloc
  else {
    pthread_mutex_lock(&lock);
    new_block = sbrk(size + sizeof(Metadata));
    pthread_mutex_unlock(&lock);
  }
  if(new_block == (void*)-1) {
    return NULL;
  }
  new_block->available = 0;
  new_block->size = size;
  new_block->next = NULL;
  new_block->prev = NULL;
  return new_block;
}
```

The exact codes of locked malloc and unlocked malloc and their free() have been displayed below.

```c
void * ts_malloc_lock(size_t size) {
  pthread_mutex_lock(&lock);
  int sbrk_lock = 0;
  void * ptr = bf_malloc(size, sbrk_lock, &first_free_lock_blk, &last_free_lock_blk);
  pthread_mutex_unlock(&lock);
  return ptr;
}

void ts_free_lock(void *ptr) {
  pthread_mutex_lock(&lock);
  bf_free(ptr, &first_free_lock_blk, &last_free_lock_blk);
  pthread_mutex_unlock(&lock);
}

void * ts_malloc_nolock(size_t size) {
  int sbrk_lock = 1;
  void * ptr = bf_malloc(size, sbrk_lock, &first_free_unlock_blk, &last_free_unlock_blk);
  return ptr;
}

void ts_free_nolock(void *ptr) {
  bf_free(ptr, &first_free_unlock_blk, &last_free_unlock_blk);
}
```

2. Result and analysis

2.1 Result

In this section, I will display the result of performance experimentation and the corresponding explanations. **The result is the average of three experiments on two versions with the thread_test_measurement.c test**. The result is exhibited in the table below.

version	Execution Time / s	Data Segment Size / byte
Locking version	1.264	43,834,763
Unlocking version	0.269	43,355,093

2.2 Analysis

As the data shown in the table, the unlocking version is faster than the locking version. **The reason is that locking version will lock the entire process of malloc, which leads to the longer waiting than the unlocking version. In unlocking version, it only blocks other threads when it needs to call sbrk().** So, in other part of malloc, unlocking version allows multi-threads concurrently work together.

The data segment sizes in two versions are very similar, which means the efficiency of the reuse of allocated blocks and the merge of freed blocks in the two versions is similar.